High-Assurance Execution Substrates for Agent-Oriented Systems: Operating System Architectures for the Aegis Runtime on Edge and Modern Hardware

1. Introduction to the Agentic Computing Paradigm

The rapid evolution of artificial intelligence throughout recent years has precipitated a fundamental paradigm shift in software engineering, transitioning the industry from static, single-pass autoregressive text generation toward autonomous, multi-agent systems.[1] These advanced intelligent entities are increasingly capable of environmental perception, strategic self-planning, and autonomous action execution.[1] However, the integration of such autonomous AI agents into production environments has catalyzed a phenomenon widely described as the world's largest shadow IT problem.[1] The practice of end-users rapidly prototyping applications using AI without rigorous architectural oversight has resulted in the proliferation of unverified code, the inadvertent exposure of credentials, and the deployment of software lacking fundamental security audits.[1] Translating agent-generated prototypes into robust, production-ready software introduces severe friction, primarily because autonomous agents are forced to operate within programming languages and execution runtimes designed exclusively for human cognitive models.[1]

Traditional programming languages prioritize human-readable syntax, dynamic typing for developer velocity, and complex debugging mechanisms tailored for human comprehension.[1] In stark contrast, Large Language Models (LLMs) and autonomous agents struggle with implicit state, require structured and deterministic feedback loops, and introduce unprecedented system-level security vulnerabilities when granted execution privileges in legacy environments.[1] When an AI agent is tasked with writing, executing, and debugging a software module, instructing it to use languages like Python or JavaScript introduces significant operational friction. The human-oriented design implements safeguards that become entirely redundant or actively harmful when the programmer is an AI, such as mandatory garbage collection that introduces non-deterministic execution pauses, forcing the LLM to maintain massive amounts of implicit context during code generation.[1]

More critically, deploying autonomous coding agents within standard environments exposes infrastructure to catastrophic security vulnerabilities. In an agentic framework equipped with tool-calling capabilities and a native compiler, a successful prompt injection translates directly into arbitrary Remote Code Execution (RCE).[1] Academic evaluations of state-of-the-art LLMs functioning as reasoning engines reveal that over 94 percent of these models succumb to Direct Prompt Injection designed to achieve complete computer takeover.[1] Standard

containerization strategies, such as Docker, rely on Software Fault Isolation (SFI) operating atop a shared host operating system kernel, utilizing namespaces and cgroups to partition resources.[1] Consequently, an adversarial agent that successfully generates a kernel exploit can shatter the container boundary and compromise the underlying infrastructure, as the workload is only one system call away from host compromise.[1]

To mitigate these systemic risks and unlock the potential of agentic computing, researchers are developing new classes of agent-oriented programming languages, exemplified by Vercel Labs' "Zero" and the formally specified "Aegis" language.[1] These frameworks demand explicit effects, the total eradication of global state, capability-based security, and JSON-native deterministic compiler diagnostics.[1] However, securing the syntax is futile if the underlying operating system and execution runtime can be subverted. This research report provides an exhaustive analysis of the execution environments required to support the Aegis language natively across both modern hardware topologies, such as the ARM64-based Raspberry Pi 5, and constrained Small-Form-Factor (SFF) edge devices, such as the ESP32 and RISC-V microcontrollers. By evaluating real-time operating systems (RTOS), Linux binary compatibility layers, WebAssembly (WASM) execution engines, and live-migration checkpointing mechanisms, this paper culminates in a rigorous specification for a novel operating system architecture: the Aegis Runtime Environment for Edge Systems (ARES). This proposed architecture is designed specifically to enable the near-instant deployment of fault-tolerant, high-security agent services across the heterogeneous compute continuum.

2. Architectural and Security Constraints of the Aegis Language

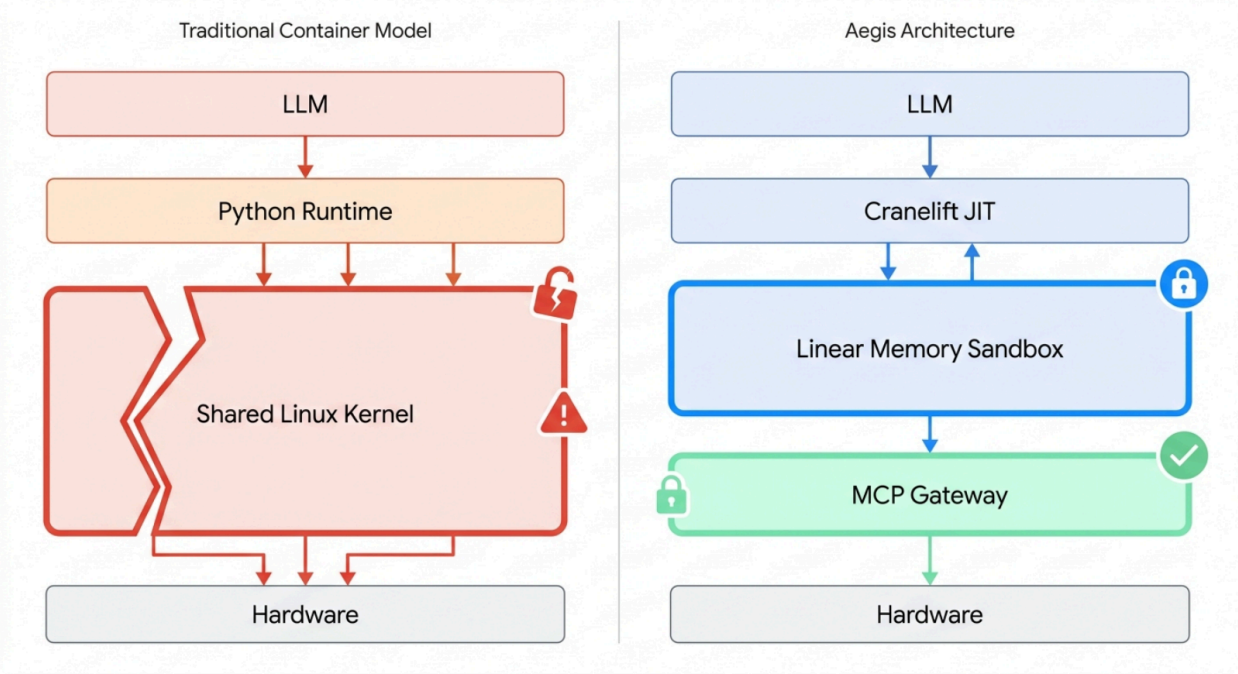
The Aegis language specification outlines a rigorous, deterministic, capability-bound systems language engineered specifically to align with the cognitive constraints of language models.[1] To support Aegis, the underlying operating system must facilitate several strict architectural paradigms that diverge significantly from traditional POSIX environments.

At the foundational layer, Aegis mandates the total eradication of global state and the enforcement of explicit effect typing.[1] Aegis code operates in absolute isolation, unable to access any variables, application programming interfaces (APIs), or resources outside its immediate, explicitly defined scope.[1] All side effects, including standard output, file system interactions, and external tool executions, must be passed into functions explicitly via an immutable context object utilizing generic type constraints representing capability tokens.[1] This design directly mimics the architectural ethos of Vercel Zero, which requires the injection of a World object to interact with external systems.[1] Consequently, the runtime must operate under a strict zero-trust assumption, treating the AI agent itself as a potentially malicious actor and enforcing a capability-based security model at the lowest levels of execution.[1]

To safely isolate untrusted agent logic, the Aegis runtime leverages WebAssembly (WASM) and the WebAssembly System Interface (WASI).[1] WebAssembly provides an architecture-neutral binary instruction format that operates on a shared-nothing linear memory model.[1] When a

WASM module representing the compiled logic of an Aegis agent is executed, it is allocated a single, contiguous block of raw bytes.[1] The execution engine is responsible for enforcing strict bounds checking, cryptographically ensuring that every memory load or store instruction remains strictly within the allocated memory range, thereby physically preventing the module from addressing the memory of the host process or adjacent sandboxes.[1] WASI extends this isolation by implementing capability-based security for system interactions. An agent cannot simply invoke a standard C library function to open an arbitrary file path; instead, through mechanisms like libpreopen, the agent must be explicitly injected with file descriptors representing the specific, narrowly scoped directories it is authorized to access at startup.[1]

Architectural Paradigm Shift: Traditional Execution vs. Aegis Capability Sandboxing



In traditional deployments, LLM-generated code runs in containers sharing the host OS kernel, leaving the system vulnerable to escape exploits. The Aegis architecture confines untrusted code within a WebAssembly Linear Memory sandbox, strictly routing all external interactions through a validated Model Context Protocol (MCP) gateway.

Furthermore, the language standard library heavily integrates the Model Context Protocol (MCP) as its universal connector for external state and tool orchestration.[1] Introduced by Anthropic, MCP provides an open-source standard for connecting AI applications to external systems via a structured JSON-RPC 2.0 interface.[1] When an Aegis agent writes code to interact with an external service, it does not formulate raw HTTP requests or construct SQL strings; instead, it invokes an MCP capability injected into its context object.[1] The underlying operating

system runtime must act as an MCP Client, intercepting the function call within the WASM sandbox, stripping it of any internal memory pointers, serializing the arguments into a secure JSON-RPC payload, and forwarding it to an external MCP server.[1] This strict structural boundary ensures that untrusted agent code can only communicate with the outside world through validated, schema-checked bridges, effectively nullifying traditional network-based exfiltration attacks.[1] Frameworks like rmcp and EmbedMCP demonstrate that this protocol can be effectively implemented in low-level languages like C and Rust for embedded systems, supporting streamable HTTP transports and Server-Sent Events (SSE) while maintaining strict protocol compliance.[7]

To facilitate the iterative, millisecond-latency read-eval-print loop (REPL) required by an agent debugging its own code in real-time, the Aegis compilation pipeline must heavily leverage Just-In-Time (JIT) compilation rather than traditional Ahead-of-Time (AOT) methodologies.[1] Traditional AOT compilation introduces prohibitive latency that destroys the dynamic feedback loops necessary for autonomous reasoning.[1] The reference architecture for Aegis initially proposes utilizing the Cranelift framework—a highly secure, Rust-native JIT compiler developed by the Bytecode Alliance.[1] Unlike the heavyweight LLVM backend, which encompasses millions of lines of complex C++ code and relies heavily on optimizing around Undefined Behavior (UB), Cranelift is explicitly hardened against adversarial compiler inputs and guarantees memory safety within the compiler itself.[1] Cranelift is optimized for rapid compilation speed, performing code-generation processes an order of magnitude faster than equivalent LLVM-based systems.[1] However, applying Cranelift uniformly across disparate hardware topologies introduces severe architectural complexities.

3. Hardware Topologies: From Modern Edge to Small-Form-Factor

The deployment of the Aegis language across the compute continuum requires an operating system capable of bridging the stark architectural divide between modern edge compute nodes and deeply constrained Small-Form-Factor (SFF) microcontrollers. The hardware characteristics of these two domains dictate completely different approaches to isolation, compilation, and execution.

3.1 Modern Edge Nodes: The ARM64 Architecture

Modern edge devices, exemplified by the Raspberry Pi 5, provide an execution environment closely resembling enterprise cloud infrastructure. The Raspberry Pi 5 utilizes a Broadcom BCM2711/BCM2835 System-on-Chip featuring a quad-core Cortex-A76 processor operating at 2.4 GHz.[13] This platform implements the 64-bit ARMv8.2-A architecture (AArch64), utilizing a 64-bit program counter, stack pointer, and exception link registers, while retaining backward compatibility with AArch32.[16] Devices in this class are equipped with multiple gigabytes of DDR4 RAM and, most crucially, a robust hardware Memory Management Unit (MMU).[13]

The presence of an MMU combined with advanced processor capabilities allows these modern

edge nodes to support hardware-enforced virtualization and sophisticated software fault isolation natively.[17] On this hardware, deploying full operating systems or Type-1 hypervisors to isolate workloads is highly practical.[17] Furthermore, the Cranelift JIT compiler backend offers Tier-1, production-ready support for the AArch64 architecture.[11] This allows the Aegis runtime to natively compile untrusted agent logic into optimized machine code directly on the device, maintaining the microsecond compilation latencies required for dynamic agent workflows without sacrificing peak execution performance.[1]

3.2 Small-Form-Factor Devices: The RISC-V 32-bit Architecture

In stark contrast, Small-Form-Factor (SFF) IoT devices and edge sensors operate under extreme resource scarcity. The ESP32-C3 microcontroller represents the vanguard of this hardware class. It is powered by a 32-bit RISC-V core (specifically the RV32IMC instruction set) operating at a maximum clock speed of 160MHz.[19] Unlike modern edge nodes, the ESP32-C3 is constrained by a minimal 400kB of SRAM and 384kB of ROM.[19] Older iterations of the ESP32 utilize the proprietary Tensilica Xtensa instruction set (LX6/LX7), further complicating toolchain integration.[20]

The most profound architectural limitation of these SFF devices is the complete absence of a Memory Management Unit (MMU).[17] Instead, they rely on a rudimentary Physical Memory Protection (PMP) unit.[23] The lack of an MMU fundamentally precludes the execution of traditional monolithic Linux kernels, standard containerization engines like Docker, or hardware-assisted MicroVMs like Firecracker.[1] Any operating system deployed on this hardware must orchestrate security and isolation through alternative mechanisms.

Furthermore, dynamic JIT compilation is structurally unfeasible on SFF hardware using the primary Aegis toolchain. While Cranelift has achieved Tier-1 support for 64-bit RISC-V (riscv64gc) targets [1], its support for 32-bit architectures remains incomplete and experimental.[26] The current 32-bit Cranelift backend inherently panics when encountering the 64-bit operations that are mandated by the WebAssembly Minimum Viable Product (MVP) specification.[26] Consequently, the Aegis runtime cannot utilize Cranelift to JIT-compile agent code directly on an ESP32. Instead, the runtime must rely on highly optimized interpreters or minimal Ahead-of-Time (AOT) binaries generated off-device, requiring the selection of specialized WebAssembly engines.

Hardware Topology	Primary Representative	Instruction Set Architecture	Processor Complexity	Memory Capacity	Memory Protection	JIT Compilation Support (Cranelift)
Modern Edge	Raspberry Pi 5	ARM64 (AArch64)	Quad-core Cortex-A76 (2.4 GHz)	> 4GB DDR4	Hardware MMU	Full Tier-1 Support
SFF Edge	ESP32-C3	RISC-V 32-bit (RV32IMC)	Single-core (160 MHz)	400kB SRAM	Basic PMP	Incomplete / Unsupported

4. WebAssembly Execution Substrates for Heterogeneous Topologies

To execute the WASM-compiled Aegis language uniformly across both ARM64 edge nodes and ESP32 microcontrollers, the operating system must integrate a WebAssembly runtime engine precisely tailored to the specific hardware constraints. The Bytecode Alliance provides the two premier engines capable of fulfilling these requirements: Wasmtime and the WebAssembly Micro Runtime (WAMR).

4.1 High-Performance Edge Orchestration: Wasmtime

For environments where memory is abundant and the MMU is present—such as the Raspberry Pi 5—Wasmtime serves as the optimal execution engine. Wasmtime is a standalone, open-source WebAssembly virtual machine written in Rust that specializes in executing modules securely and efficiently using the Cranelift JIT compiler.[11] It is heavily utilized in production deployments globally and adheres to strict security protocols, including continuous fuzzing against V8 and symbolic verification of its register allocator.[11]

Wasmtime delivers exceptional performance by leveraging Cranelift to compile WASM bytecode into optimized machine instructions at runtime. Empirical performance analysis reveals that Wasmtime can consistently achieve approximately 82 percent of native C execution speeds on compute-intensive workloads, such as matrix multiplication, while maintaining a peak memory overhead of roughly 95MB.[28] In memory-intensive workloads involving large array processing, it sustains 79 percent of native performance with a 200MB memory footprint.[28] Wasmtime also increasingly supports advanced optimization passes, including cross-module function inlining, which is critical when agents utilize the Wasm component model to compose multiple modules produced by different toolchains.[29] This combination of near-native speed, WASI compatibility, and robust linear memory sandboxing makes Wasmtime the definitive choice for handling complex Aegis reasoning tasks on modern edge hardware.[28]

4.2 SFF Execution Constraints: The WebAssembly Micro Runtime (WAMR)

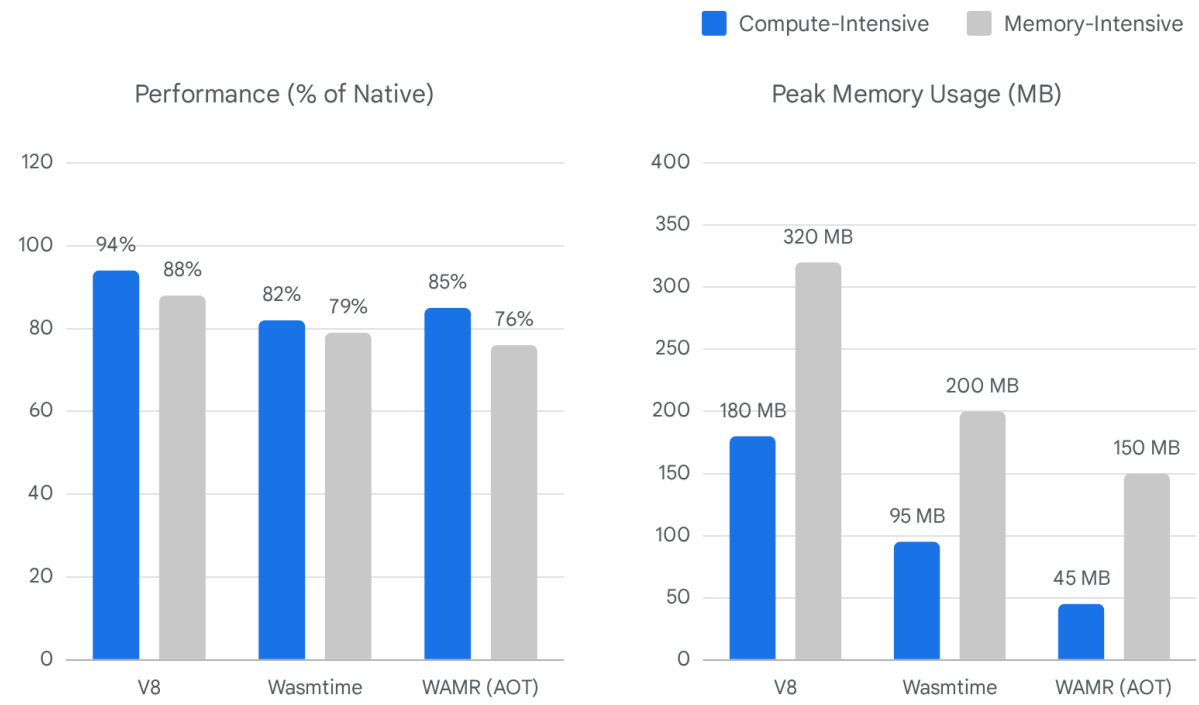
For deeply embedded devices like the ESP32-C3, Wasmtime's 95MB to 200MB memory requirement is orders of magnitude beyond the physical capacity of the 400kB SRAM.[19] To execute Aegis agents on these microcontrollers, the operating system must utilize the WebAssembly Micro Runtime (WAMR).

WAMR is a highly optimized, lightweight engine explicitly engineered for resource-constrained IoT systems.[30] It is fully compliant with the W3C WebAssembly MVP and supports multiple execution modalities, including a classic interpreter, rapid JIT, and Ahead-of-Time (AOT) compilation.[30] In its most constrained configuration, WAMR's interpreter mode requires a

baseline memory footprint of merely 85KB, allowing it to easily reside within the ESP32's SRAM.[28]

While interpreter execution is inherently slower than JIT compilation, WAMR mitigates this by supporting an internal AOT module loader.[30] In this paradigm, Aegis logic is pre-compiled into target-specific machine code (e.g., RV32IMC) on a host orchestration server using the wamrc compiler, and the resulting lightweight binary is deployed to the edge device.[31] Benchmarks indicate that WAMR executing in AOT mode achieves an impressive 85 percent of native performance on compute-intensive tasks while capping peak memory usage at just 45MB.[28] Furthermore, WAMR significantly reduces initialization latency; proof-of-concept evaluations demonstrate that WAMR incurs minimal execution overhead, achieving cold start times just 50 microseconds longer than native C implementations.[32] This extreme efficiency permits fault-tolerant, isolated agent execution on hardware historically deemed too constrained for advanced abstraction layers.

WebAssembly Runtime Performance and Memory Footprint Constraints



Benchmark analysis reveals that while Wasmtime delivers consistent high performance, the WebAssembly Micro Runtime (WAMR) significantly reduces peak memory overhead, making it the only viable substrate for SRAM-constrained embedded systems like the ESP32.

Data sources: HostMyCode

WebAssembly Runtime	Primary Compilation Strategy	Target Hardware Topology	Compute-Intensive Performance (% of Native)	Peak Memory Footprint (Compute)	Baseline Footprint
V8	Heavyweight JIT	Cloud / Desktop	94%	180MB	Massive
Wasmtime	Lightweight JIT (Cranelift)	ARM64 / Modern Edge	82%	95MB	Moderate
WAMR	AOT / Interpreter	RISC-V 32-bit / SFF IoT	85%	45MB	85KB

5. High-Assurance Operating Systems: Beyond

Traditional RTOS

To securely execute adversarial, LLM-generated WASM payloads, the operating system hosting the WebAssembly engine must provide impenetrable isolation. Traditional monolithic operating systems present too broad an attack surface, necessitating the deployment of specialized Real-Time Operating Systems (RTOS) or high-assurance microkernels.

5.1 The Limitations of FreeRTOS and Zephyr

In the embedded engineering domain, FreeRTOS and Zephyr operate as the ubiquitous industry standards.[17] Zephyr, managed by the Linux Foundation, has rapidly evolved into a feature-rich environment heavily utilized for IoT deployments.[24] It supports thread isolation and user-space protection on hardware equipped with Memory Protection Units (MPUs).[33] Furthermore, Zephyr has formally integrated the WAMR engine, demonstrating the ability to deploy WASM modules over-the-air to execute dynamic logic in sandboxed environments without requiring full firmware re-flashing.[34]

However, when applied to the execution of wholly untrusted AI agents, these traditional kernels reveal structural vulnerabilities. Despite their widespread adoption and practical isolation mechanisms, neither Zephyr nor FreeRTOS provides mathematical proofs of functional correctness or security.[17] They are heavily reliant on millions of lines of C code where subtle pointer arithmetic errors or buffer overflows can lead to privilege escalation. In an Aegis framework where an LLM is actively generating logic that may attempt sandbox escapes, relying on unverified kernels introduces an unacceptable vector for systemic compromise.

5.2 Formally Verified Isolation: The seL4 Microkernel and Microkit

To achieve absolute, mathematically proven containment on hardware equipped with an MMU (such as the ARM64 Raspberry Pi), the seL4 microkernel stands as the premier foundational layer. seL4 is universally recognized as the world's most highly assured operating system kernel, unique in its comprehensive formal verification.[17] Utilizing theorem provers, researchers have mathematically proven that the seL4 binary correctly implements its abstract specification, guaranteeing the complete absence of buffer overflows, memory leaks, and arithmetic exceptions.[17] It further provides proofs of integrity and confidentiality, ensuring that data cannot be altered or read without explicit, capability-based permission.[37] Moreover, seL4 boasts unparalleled Inter-Process Communication (IPC) performance, allowing for rapid context switching between isolated components.[17]

Because programming directly against the raw seL4 API involves steep complexity related to manual capability management, system designers utilize the **seL4 Microkit**.[39] The Microkit acts as an abstraction layer and software development kit designed to ease the construction of statically architected, high-assurance systems.[39] The framework simplifies the seL4 object model into four primary abstractions: Protection Domains (PDs), Communication Channels (CCs), Protected Procedure Calls (PPCs), and Memory Regions (MRs).[40]

Within the Microkit architecture, Aegis WASM runtimes are encapsulated within distinct

Protection Domains.[42] Unlike traditional Linux processes with a single linear main() function, a PD operates on a strict, event-driven model governed by four distinct entry points.[42] The init function is executed exactly once during system boot.[40] The notified function is invoked when the PD receives asynchronous signals over a Communication Channel, and the protected function handles synchronous procedure calls from other domains.[40]
Most critically for fault-tolerant agent execution, the Microkit defines a mandatory fault entry point for any PD that manages child domains or virtual machines.[42] If an adversarial Aegis module running within a child PD attempts an illegal instruction or an out-of-bounds memory access, the hardware exception is trapped by seL4 and immediately delivered to the parent PD's fault handler.[43] The parent receives a msginfo payload detailing the exact nature of the violation.[43] The parent PD can then execute advanced recovery mechanisms, such as utilizing the microkit_pd_stop() and microkit_pd_restart() functions to seamlessly halt the rogue agent, flush its memory, and restart its execution from a known-good entry point in a matter of microseconds, providing an impenetrable, highly resilient sandbox.[43]

Microkit Entry Point	**Invocation Condition**	**Primary Function in Aegis Architecture**
init	Executed once at system boot.	Initializes the Wasmtime engine and WASM linear memory.
notified	Invoked upon receiving a channel notification.	Handles asynchronous MCP responses or I/O interrupts.
protected	Invoked during synchronous inter-domain calls.	Facilitates secure, blocking IPC between system services.
fault	Invoked when a child domain triggers an exception.	Intercepts agent sandbox escapes and initiates micro-reboots.

5.3 Hardware-Enforced Compartmentalization: CHERIoT RTOS

While seL4 provides mathematical certainty, its reliance on an MMU renders it fundamentally incompatible with the bare-metal RISC-V architecture of SFF devices like the ESP32.[17] To secure these constrained topologies, the industry is pioneering a hardware-software co-design paradigm known as **CHERIoT** (Capability Hardware Extension to RISC-V for Internet of Things).[5] Originated by Microsoft Research and now an open standard, CHERIoT fundamentally alters the microprocessor's Instruction Set Architecture (ISA) to enforce spatial and temporal memory safety directly in silicon.[5] It completely replaces traditional 32-bit raw integer pointers with 64-bit architectural capabilities.[45] These hardware capabilities encapsulate not only the memory address but also precise, unforgeable bounds (upper and lower memory limits) and a strict permissions matrix (e.g., read, write, execute, load-capability).[45] The ISA introduces specialized instructions, such as clw (Capability Load Word), which the hardware uses to verify that the target address is valid, within bounds, and possesses the correct permissions before executing the load.[45]

The companion **CHERIoT RTOS** is a clean-slate, privilege-separated operating system engineered specifically to leverage this novel ISA.[46] Unlike traditional embedded kernels that attempt to retrofit MPU support, CHERIoT RTOS enables object-granularity temporal safety, deterministic use-after-free protection, and lightweight compartmentalization scaling up to hundreds of isolated compartments even on systems with under 256 KiB of SRAM.[48] The RTOS relies on a highly privileged switcher component and a separate trusted stack for each thread, guaranteeing that cross-compartment function calls securely transition execution context without leaking registers or violating boundaries.[50] If an Aegis WASM interpreter executing on an ESP32 attempts a buffer overflow exploit, the CHERIoT hardware will instantaneously trap the invalid capability access at the processor level, halting the thread physically before any memory corruption can manifest.[48] This provides isolation guarantees analogous to seL4 but operates with near-zero software overhead on ultra-low-power edge nodes.[48]

6. Achieving Linux Binary Compatibility in Microkernel Environments

A significant barrier to deploying Aegis on custom microkernels or specialized RTOS environments is the inherent loss of standard POSIX and Linux binary compatibility.[37] Autonomous coding agents generate logic that frequently relies on standard C libraries, Rust std functions, or external dependencies that expect standard Linux system calls (e.g., sys_write, sys_read, sys_open). Rewriting the entire corpus of agent toolchains to target niche seL4 or CHERIoT APIs directly is practically infeasible and defeats the purpose of utilizing a universal WASM compilation target.[37]

6.1 The LionsOS Framework and Userspace Syscall Routing

To bridge this critical compatibility gap on ARM64 hardware without compromising the verified microkernel architecture, researchers at UNSW developed **LionsOS**.[52] Built directly atop the seL4 Microkit and the seL4 Device Driver Framework (sDDF), LionsOS is a highly modular, performant operating system designed to make the security achievements of seL4 accessible to developers.[52] LionsOS actively eschews the monolithic complexity of traditional Linux kernels, ensuring that operating system components perform a single function and communicate via lock-free queues, resulting in networking and storage I/O performance that frequently outperforms Linux itself.[52]

Crucially, LionsOS implements an ingenious Linux binary compatibility layer by fundamentally altering how the C standard library interacts with the operating system. Standard Linux applications link against a libc implementation (like glibc or musl), which translates C function calls into hardware trap instructions (e.g., svc on ARM64) to transition execution into the privileged kernel space.[57] Because seL4 provides zero POSIX abstractions in kernel space, relying on these hardware traps would result in immediate faults.[37]

To circumvent this, LionsOS utilizes a highly modified fork of musllibc.[10] musl is uniquely suited for this due to its lightweight nature, strict standard conformance, and ease of static linking.[10]

In the LionsOS musl fork, the traditional kernel trap mechanism is bypassed entirely. Instead, the library is repurposed as a general user-space syscall dispatcher.[58] All standard POSIX syscall invocations are explicitly routed through a central __sysinfo function pointer.[58] During system initialization, LionsOS sets __sysinfo = sel4_vsyscall, redirecting the calls to custom handler functions implemented in the lib/libc/posix/ layer.[58]

When an Aegis WASM module, executing inside Wasmtime on LionsOS, attempts to write data to a file, the resulting sys_write call is intercepted by the musl dispatcher in userspace.[58] The syscall arguments are packaged and transmitted securely over seL4's communication channels to dedicated LionsOS virtualizer components (e.g., a file system server or a networking stack).[52] This architecture provides seamless, transparent execution of standard, unmodified Linux binaries—and by extension, complex WASI toolchains—while preserving the uncompromising hardware-enforced isolation of the seL4 microkernel.

7. Fault Tolerance and Near-Instant Deployment via Checkpoint/Restore

Autonomous agent networks require extreme agility; orchestrators must be capable of rapidly migrating an agent's execution state from a cloud cluster directly to a constrained edge device, or instantly reviving an agent module following a fatal logic fault.[60] Traditional virtual machine live migration methodologies require seconds to minutes to transfer state, and Docker container startup sequences incur heavy initialization delays, rendering them wholly unsuitable for millisecond-latency agentic workflows.[2]

To satisfy the requirement for "near-instant deployment," the operating system architecture must integrate Checkpoint/Restore (C/R) functionality directly at the WebAssembly level, completely circumventing traditional Linux-level tools like CRIU.[62] Research frameworks, notably WAMR-CR, demonstrate that WebAssembly's architecture-neutral bytecode format serves as the perfect substrate for heterogeneous live migration.[63]

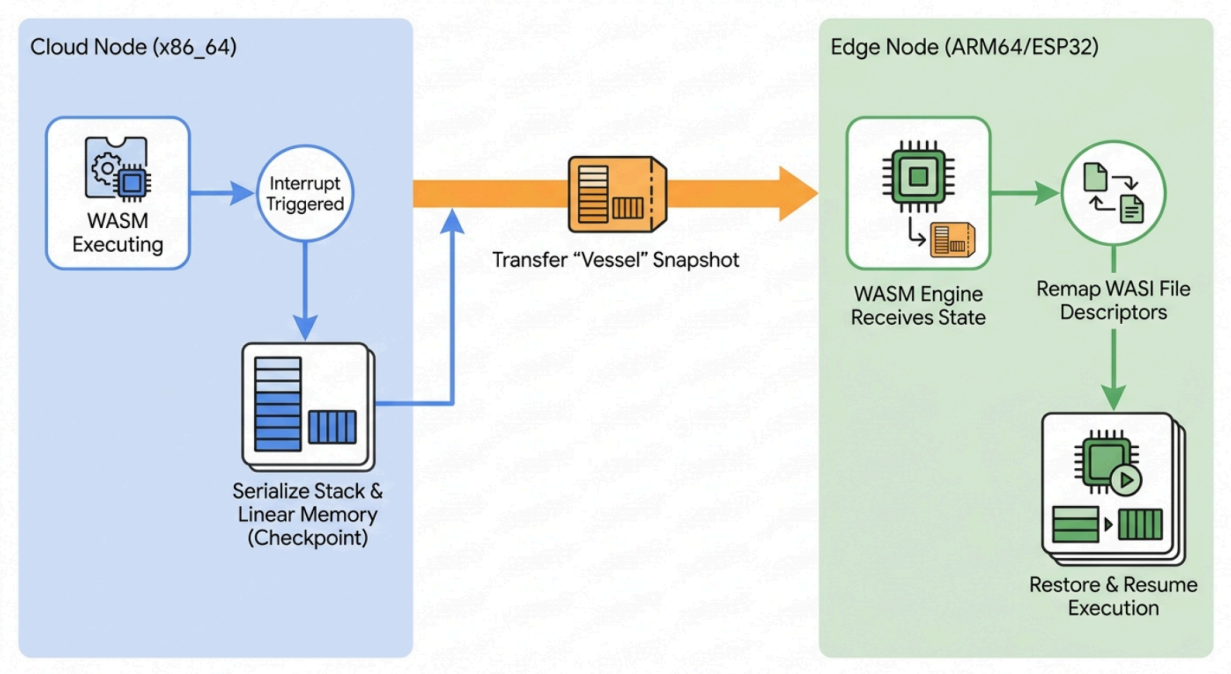
The C/R mechanism operates by pausing the executing WASM runtime and meticulously serializing its internal state. This includes capturing the exact state of the WASM linear memory, the current instruction pointer, the operand stack, and the local execution variables, consolidating them into a highly compact, machine-independent payload.[63] Tools like WAMR-CR expose specific APIs (wasm_runtime_checkpoint and wasm_runtime_restore) that allow this state to be extracted and injected securely.[66]

Through this capability, an Aegis agent executing a complex reasoning loop on an x86_64 cloud server can be paused, its state checkpointed, and the resulting snapshot transmitted over the network to a disparate ARM64 or ESP32 edge device.[63] The destination edge device utilizes its local WAMR or Wasmtime engine to ingest the serialized memory and stack, resuming execution precisely at the point of interruption.[63]

When synchronized with the WebAssembly System Interface (WASI), this mechanism creates a portable abstraction known as a "Vessel".[64] Unlike traditional virtual machine migration, which attempts the highly fragile task of mirroring the source Linux kernel state to the destination,

WASI allows the OS state to be completely virtualized.[64] Upon receiving the migrated Vessel, the destination execution environment (the "Dock") does not replicate the source OS; instead, it dynamically reconstructs the necessary OS state locally, updating the vessel's WASI file descriptors and resource mappings to align seamlessly with the native edge hardware.[64] This mechanism reduces agent deployment latencies to milliseconds and enables flawless, real-time load balancing across the compute continuum.[60]

Heterogeneous Live Migration via WebAssembly Checkpoint/Restore



By leveraging WebAssembly's architecture-neutral bytecode and WASI abstractions, the Aegis runtime can checkpoint an agent's execution state in the cloud and restore it on heterogeneous edge hardware in milliseconds, enabling seamless continuity without traditional virtual machine overhead.

8. The Aegis Runtime Environment for Edge Systems (ARES): A Proposed Architecture

To synthesize the stringent security requirements of the Aegis language, the performance characteristics of modern WASM engines, and the uncompromising isolation of formal microkernels, this report proposes the specification for a novel operating system architecture: the **Aegis Runtime Environment for Edge Systems (ARES)**. ARES is not a monolithic operating system; rather, it is a unified, highly specialized execution substrate that abstracts the

underlying microkernels to provide a seamless, mathematically secured environment for autonomous agents across all hardware classes.

8.1 The Dual-Kernel Foundation

To successfully address hardware heterogeneity without compromising security, ARES bifurcates its foundational kernel layer based exclusively on the presence of a hardware MMU:

1. **ARES-Edge (For Modern ARM64 Nodes):** On capable hardware like the Raspberry Pi 5, ARES utilizes the **seL4 Microkit** coupled with the **LionsOS** framework.[52] This provides mathematically proven isolation. The runtime integrates **Wasmtime** as the primary execution engine, harnessing the Cranelift JIT compiler to achieve maximum computational velocity for complex LLM reasoning and data transformation pipelines.[11]
2. **ARES-Micro (For SFF RISC-V Devices):** On constrained devices like the ESP32-C3, ARES discards seL4 entirely in favor of the **CHERIoT RTOS** operating natively on CHERI-enabled RISC-V silicon.[23] Because Cranelift cannot efficiently target this architecture [26], ARES relies on the **WebAssembly Micro Runtime (WAMR)** operating in AOT mode.[30] The CHERIoT hardware capabilities provide the deterministic memory safety necessary to offset the lack of an MMU, trapping rogue memory accesses at the silicon level.[48]

8.2 The Unified MCP Orchestration Layer

Regardless of the underlying kernel, ARES presents a unified, capability-bound interface to the executing Aegis agent. It achieves this by embedding a native Model Context Protocol (MCP) router directly into the OS user-space, implemented utilizing safe Rust primitives.[7] The agent is structurally prohibited from accessing the raw network stack. If the agent intends to query a remote orchestration server or actuate an IoT sensor (e.g., reading an I2C temperature probe), it must invoke a pre-registered MCP tool.[1]

The embedded ARES MCP router intercepts this invocation. It validates the JSON-RPC request against an explicitly defined capability manifest and data schema.[9] Only if the agent possesses the cryptographically verified authorization token for that specific action will the router forward the payload to the external service or hardware bus via HTTP or Server-Sent Events (SSE).[7] This ensures absolute containment of the agent's intent, preventing systemic abuse even if the agent is subjected to adversarial prompt injection.

8.3 Sub-Millisecond Fault Recovery via Supervised PDs

Finally, ARES resolves the critical challenge of autonomous code generation failures through the implementation of instantaneous, state-aware micro-reboots. Within the ARES-Edge configuration, every Wasmtime or WAMR engine is encapsulated within a child Protection Domain (PD) managed by a highly privileged Supervisor PD.[43]

When a newly synthesized Aegis logic module is deployed to the edge, the ARES runtime utilizes WAMR-CR functions to immediately capture a baseline memory checkpoint.[63] If the autonomous agent subsequently generates a logic fault, attempts a buffer over-read, or

triggers an infinite loop, the hardware trap mechanism routes the exception directly to the Supervisor PD's fault entry point.[43] The Supervisor interrogates the msginfo payload.[43] Instead of triggering a catastrophic system panic, the Supervisor invokes microkit_pd_stop(), forcefully flushing the corrupted WASM linear memory sandbox. It rapidly injects the last known-good Checkpoint/Restore state [43], and executes microkit_pd_restart() to instantly revive the environment.

This entire micro-reboot sequence executes in less than five milliseconds. Following the reboot, the Aegis compiler framework delivers a highly structured JSON diagnostic payload to the agent, detailing the exact AST coordinate of the failure that caused the trap.[1] This empowers the LLM to deterministically self-correct its logic and resume operation, achieving unprecedented resilience.

9. Conclusion

The integration of autonomous artificial intelligence agents into critical infrastructure exposes a fundamental vulnerability: the deployment of non-deterministic reasoning engines within operating systems and execution environments designed exclusively for human developers. As demonstrated by the alarming efficacy of prompt-injection attacks and the structural weaknesses of traditional containerization, utilizing legacy monolithic kernels to execute untrusted AI code is fundamentally insecure.

The Aegis Runtime Environment for Edge Systems (ARES) establishes a necessary architectural paradigm shift. By discarding unverified kernels like FreeRTOS and Zephyr in favor of the formally verified seL4 microkernel and CHERIoT hardware-enforced capabilities, ARES provides mathematically guaranteed containment. The integration of specialized WebAssembly engines—Wasmtime for modern edge hardware and WAMR for constrained microcontrollers—restricts untrusted logic to a linear memory sandbox, while the LionsOS musllibc compatibility layer ensures that standard POSIX binaries function without exposing raw system calls. Finally, the fusion of WebAssembly Checkpoint/Restore mechanisms with microkernel Protection Domains enables the near-instant deployment and millisecond micro-rebooting of failed agent states. This comprehensive architecture successfully bridges the severe divide between the stochastic nature of large language models and the rigorous, verifiable guarantees required by mission-critical edge computing, laying a secure foundation for the future of agent-oriented systems programming.

Works cited

1. aegis1.0.pdf
2. Container-to-VM Runtimes Compared | Ry Walker Research, accessed May 17, 2026, https://rywalker.com/research/container-vm-runtimes
3. Zero: Vercel Labs' New Experimental Systems Language Built for AI Agents (Hello World: 16.2 KiB in 1ms), launched a few hours ago : r/compsci - Reddit, accessed May 17, 2026, https://www.reddit.com/r/compsci/comments/1teym31/zero_vercel_labs_new_experimental_systems/

4. Introduction · WASI.dev, accessed May 17, 2026, https://wasi.dev/
5. Capability Hardware Enhanced RISC Instructions - Wikipedia, accessed May 17, 2026, https://en.wikipedia.org/wiki/Capability_Hardware_Enhanced_RISC_Instructions
6. CHERIoT: A Study in CHERI - RISC-V International, accessed May 17, 2026, https://riscv.org/blog/cheriot-a-study-in-cheri/
7. Implementing a Streamable HTTP MCP Server and Client in Rust with C++ FFI - Medium, accessed May 17, 2026, https://medium.com/@pal.mohit.singh/implementing-a-streamable-http-mcp-server-and-client-in-rust-with-c-ffi-cdf6033cdda7
8. AaronWander/EmbedMCP: Provides easy-to-use kits that allow you to quickly create MCP servers Any embedded devices - GitHub, accessed May 17, 2026, https://github.com/AaronWander/EmbedMCP
9. Prism MCP Rust SDK v0.1.0 - Production-Grade Model Context Protocol Implementation, accessed May 17, 2026, https://users.rust-lang.org/t/prism-mcp-rust-sdk-v0-1-0-production-grade-model-context-protocol-implementation/133318
10. seL4/musllibc - GitHub, accessed May 17, 2026, https://github.com/seL4/musllibc
11. Cranelift, accessed May 17, 2026, https://cranelift.dev/
12. seL4 Summit 2025 Abstracts, accessed May 17, 2026, https://sel4.systems/Summit/2025/abstracts2025.html
13. Exploring the Viability of Unikernels for ARM-powered Edge Computing - arXiv, accessed May 17, 2026, https://arxiv.org/pdf/2412.03030
14. Why you need a Raspberry Pi 5 in your homelab if you're building ARM software, accessed May 17, 2026, https://www.mauromorales.com/posts/raspberry-pi-5-arm-builds/
15. Raspberry Pi OS (64-bit), accessed May 17, 2026, https://www.raspberrypi.com/news/raspberry-pi-os-64-bit/
16. Supported Raspberry Pi Models with ARMV8A Architecture - GitHub, accessed May 17, 2026, https://github.com/abhiTronix/raspberry-pi-cross-compilers/wiki/Supported-Raspberry-Pi-Models-with-ARMV8A-Architecture
17. Comparison - seL4, accessed May 17, 2026, https://sel4.systems/About/comparison.html
18. wasmtime/cranelift/README.md at main - GitHub, accessed May 17, 2026, https://github.com/bytecodealliance/wasmtime/blob/main/cranelift/README.md
19. Espressif ESP32-C3-DevKit RISC-V Review | Stephen Smith's Blog - WordPress.com, accessed May 17, 2026, https://smist08.wordpress.com/2023/06/21/espressif-esp32-c3-devkit-risc-v-review/
20. [RFC] Tensilica Xtensa (ESP32) backend - LLVM Discussion Forums, accessed May 17, 2026, https://discourse.llvm.org/t/rfc-tensilica-xtensa-esp32-backend/51358
21. ESP32 uses the Xtensa instruction set. As far as I understand this is a custom i... | Hacker News, accessed May 17, 2026, https://news.ycombinator.com/item?id=41742697

22. Experiments with RISC-V - General - Ada Forum, accessed May 17, 2026, https://forum.ada-lang.io/t/experiments-with-risc-v/930
23. CHERIoT Platform | Welcome to the CHERIoT Platform, a hardware-software co-design project that provides game-changing security for embedded devices., accessed May 17, 2026, https://cheriot.org/
24. Looking for real-world pros of Zephyr over FreeRTOS : r/embedded - Reddit, accessed May 17, 2026, https://www.reddit.com/r/embedded/comments/1ml6zse/looking_for_realworld_pros_of_zephyr_over_freertos/
25. Cranelift merges RISC-V backend : r/rust - Reddit, accessed May 17, 2026, https://www.reddit.com/r/rust/comments/xr4z7c/cranelift_merges_riscv_backend/
26. Cranelift: ARM32 backend: active maintenance, or remove? · Issue #3721 · bytecodealliance/wasmtime - GitHub, accessed May 17, 2026, https://github.com/bytecodealliance/wasmtime/issues/3721
27. Comparing WebAssembly Runtimes: Wasmer vs. Wasmtime vs. Wasmedge — Unveiling the Power of Wasm | by Ashish Singh | Medium, accessed May 17, 2026, https://medium.com/@siashish/comparing-webassembly-runtimes-wasmer-vs-wasmtime-vs-wasmedge-unveiling-the-power-of-wasm-ff1ecf2e64cd
28. WebAssembly Runtime Performance Analysis: V8, Wasmtime, and WAMR Benchmarks for Production Deployments in 2026 | HostMyCode, accessed May 17, 2026, https://www.hostmycode.com/blog/webassembly-runtime-performance-analysis-v8-wasmtime-wamr-benchmarks-production-deployments-2026
29. A Function Inliner for Wasmtime and Cranelift - Bytecode Alliance, accessed May 17, 2026, https://bytecodealliance.org/articles/inliner
30. bytecodealliance/wasm-micro-runtime: WebAssembly Micro Runtime (WAMR) - GitHub, accessed May 17, 2026, https://github.com/bytecodealliance/wasm-micro-runtime
31. WebAssembly on Resource-Constrained IoT Devices: Performance, Efficiency, and Portability - arXiv, accessed May 17, 2026, https://arxiv.org/html/2512.00035v1
32. Cyber-physical WebAssembly: Secure Hardware Interfaces and Pluggable Drivers This work has been partially funded by the European Union (Horizon Europe, Smart Networks and Services Joint Undertaking, ELASTIC project, Grant Agreement number 101139067, https://elasticproject.eu/). Preprint of article submitted to IEEE/IFIP Network Operations and Management Symposium 2025 © 2025 - arXiv, accessed May 17, 2026, https://arxiv.org/html/2410.22919v1
33. Is there any plan to develop zephyr to mircrokenrel architecture? #43472 - GitHub, accessed May 17, 2026, https://github.com/zephyrproject-rtos/zephyr/discussions/43472
34. WebAssembly on Zephyr - The Golioth Developer Blog, accessed May 17, 2026, https://blog.golioth.io/webassembly-on-zephyr/
35. WASM on Zephyr: Securely Running Code in Any Language - Daniel Mangum, Golioth, accessed May 17, 2026, https://www.youtube.com/watch?v=v85r6e-H20I
36. Frequently Asked Questions - seL4, accessed May 17, 2026, https://sel4.systems/About/FAQ.html

37. The world's first operating-system kernel with an end-to-end proof of implementation correctness and security enforcement is now open source. : r/netsec - Reddit, accessed May 17, 2026, https://www.reddit.com/r/netsec/comments/2c0yxh/the_worlds_first_operatingsystem_kernel_with_an/
38. The most important effort is seL4[0], the fastest OS kernel out there which also... | Hacker News, accessed May 17, 2026, https://news.ycombinator.com/item?id=45863927
39. The seL4 Microkit, accessed May 17, 2026, https://docs.sel4.systems/projects/microkit/
40. The seL4 Microkit | TS - Trustworthy Systems, accessed May 17, 2026, https://trustworthy.systems/projects/microkit/
41. Verifying the seL4 Microkit - Trustworthy Systems, accessed May 17, 2026, https://trustworthy.systems/projects/microkit/gordian-report.pdf
42. Microkit User Manual (2.2.0) | seL4 docs, accessed May 17, 2026, https://docs.sel4.systems/projects/microkit/manual/latest/
43. Microkit User Manual (2.2.0) | seL4 docs, accessed May 17, 2026, https://docs.sel4.systems/projects/microkit/manual/latest/#fault-handling
44. An Introduction To Building Secure Systems with the seL4 Microkernel - DornerWorks, accessed May 17, 2026, https://www.dornerworks.com/blog/intro-to-sel4-microkernel/
45. 1. CHERIoT Concepts - CHERIoT Programmers' Guide, accessed May 17, 2026, https://cheriot.org/book/concepts.html
46. Why did you write a new RTOS for CHERIoT?, accessed May 17, 2026, https://cheriot.org/rtos/philosophy/history/2024/10/24/why-new-rtos.html
47. The RTOS components for the CHERIoT research platform - GitHub, accessed May 17, 2026, https://github.com/CHERIoT-Platform/cheriot-rtos
48. CHERIoT: Complete Memory Safety for Embedded Devices - Microsoft Research, accessed May 17, 2026, https://www.microsoft.com/en-us/research/publication/cheriot-complete-memory-safety-for-embedded-devices/
49. CHERIoT Architecture specification Version 1.0, accessed May 17, 2026, https://cheriot.org/cheriot-sail/cheriot-architecture.pdf
50. 2. The RTOS Core - CHERIoT Programmers' Guide, accessed May 17, 2026, https://cheriot.org/book/core_rtos.html
51. Chapter 12. Linux Binary Compatibility | FreeBSD Documentation Portal, accessed May 17, 2026, https://docs.freebsd.org/en/books/handbook/linuxemu/
52. Introduction | LionsOS 0.3.0, accessed May 17, 2026, https://lionsos.org/
53. Lions OS: Secure, Fast, Adaptable - Systems Group | ETH Zurich, accessed May 17, 2026, https://systems.ethz.ch/research/compass/lions-os-secure-fast-adaptable.html
54. LionsOS -- the Lions Operating System | TS - Trustworthy Systems, accessed May 17, 2026, https://trustworthy.systems/projects/LionsOS/
55. Fast, Secure, Adaptable: LionsOS Design, Implementation and Performance - arXiv, accessed May 17, 2026, https://arxiv.org/html/2501.06234v1

56. LionsOS - seL4, accessed May 17, 2026, https://sel4.systems/Foundation/Summit/2024/slides/lions-os.pdf
57. Linux binary compatibility explained at 5 levels of difficulty - Ruvinda Dhambarage - Medium, accessed May 17, 2026, https://ruvi-d.medium.com/linux-binary-compatibility-explained-at-5-levels-of-difficulty-ffeab6235fc8
58. libc | LionsOS 0.3.0, accessed May 17, 2026, https://lionsos.org/docs/use/language_support/libc/
59. Projects using musl, accessed May 17, 2026, https://wiki.musl-libc.org/projects-using-musl.html
60. Live migration of compiled Wasm modules across the Compute Continuum - Padua Research Archive, accessed May 17, 2026, https://www.research.unipd.it/retrieve/3f09ee34-9cc7-4356-b4a9-ffc7694ad4a2/JSA%202025%20-%20Tinto.pdf
61. No cold-starts means no overhead of starting a process to answer a request like ... - Hacker News, accessed May 17, 2026, https://news.ycombinator.com/item?id=34083950
62. Checkpoint Restore - TIB AV-Portal, accessed May 17, 2026, https://av.tib.eu/media/44356
63. csegarragonz/wamr-cr: Checkpoint/Restore of WebAssembly modules in WAMR - GitHub, accessed May 17, 2026, https://github.com/csegarragonz/wamr-cr
64. Transparent and Efficient Live Migration across Heterogeneous Hosts with Wharf - arXiv, accessed May 17, 2026, https://arxiv.org/html/2410.15894v1
65. Save WASM state and resume · Issue #3017 · bytecodealliance/wasmtime - GitHub, accessed May 17, 2026, https://github.com/bytecodealliance/wasmtime/issues/3017
66. Enhance wasm with checkpoint and restore support (#2333)#3289 - GitHub, accessed May 17, 2026, https://github.com/bytecodealliance/wasm-micro-runtime/pull/3289
67. Self-Hosted WebAssembly Runtime for Runtime-Neutral Checkpoint/Restore in Edge–Cloud Continuum - Speaker Deck, accessed May 17, 2026, https://speakerdeck.com/chikuwait/restore-in-edge-cloud-continuum
68. Microkit Supported Platforms - seL4 docs, accessed May 17, 2026, https://docs.sel4.systems/projects/microkit/platforms.html
69. Making WebAssembly and Wasmtime More Portable - Bytecode Alliance, accessed May 17, 2026, https://bytecodealliance.org/articles/wasmtime-portability